

Chapter 1: How Computers Work

From a transistor to an iPhone

Amir Farbin

Contents

1	How Computers Work	1
1.1	Abstraction	1
1.1.1	Working above and below the barrier	2
1.2	From Charge to Bits	2
1.2.1	The transistor	4
1.3	Logic Gates	5
1.3.1	Walking the current through a gate	6
1.3.2	The complete set	6
1.4	Representation	7
1.4.1	Binary numbers	7
1.5	Arithmetic from Logic	8
1.6	Memory	10
1.7	From a Calculator to a Computer	10
1.8	How Chips Are Made	12
1.8.1	Caches and the memory bottleneck	12
1.9	Floating Point	13
1.10	Storage	14
1.10.1	Swap	14
1.11	File Systems	14
1.12	Bootting Up	15
1.13	Operating Systems	16
1.13.1	Multitasking	16
1.14	Unix	16
1.14.1	The family tree	16
1.14.2	The Unix philosophy	17
1.15	Networks	17
1.16	Supercomputers and the Scale of Computing	18
1.17	Running One System Inside Another	19
1.18	Programming Languages	20
1.18.1	What makes a language	20
2	Glossary	23

Chapter 1

How Computers Work

From a transistor to an iPhone

In this chapter I'm going to explain a lot of things about how computers work. The purpose is general education: you're heading into an industry dominated by computing, and you should have some basic understanding of what's underneath it. I expect you know some of this already — but you've probably heard bits and pieces at best, and never put the whole picture together. This is an opportunity to do that.

One of the reasons I hope this is interesting to you is that what you're doing is fundamental computation, and your computer is your tool. Just as an artist understands their media and their techniques, this is your medium. The more you know about it, the better you'll use it. I also want to give you a gateway: you're going to confront things you've never heard of, and after this chapter, at least you'll recognize that something got mentioned once — and go look it up.

I call this “from transistor to iPhone.” We'll start with the basic ideas — electrons moving in wires — and go all the way up to a processor, an operating system, and the languages you program in.

1.1 Abstraction

Before anything else, I always make a point of introducing one concept: **abstraction**. It's probably something you all know, but it comes up constantly in this course, and in science generally. Being comfortable with abstraction is what enables you to do what you need to do.

Abstraction is the quality of dealing with ideas rather than events — the process of considering something independently of its associations, attributes, or concrete accompaniments.

What does that really mean? I need to be able to figure something out, and it can be really complicated. But once I've figured it out, I know that if I put *this* in, I get *that* out. I don't have to think about the details again. Suddenly that problem is solved, and I can go think about a different problem.

Later in this book we'll exercise this by building a checkers game piece by piece — solving individual problems that by themselves may be a little tricky or may be very simple, then

combining them. That’s how abstraction gets used in computer science.

Mathematics uses it too. There are rules behind things, and those rules may be associated with a particular physical situation. But when you pull the rules out of physical reality and understand them abstractly, you realize that completely different things may follow the same rules. Some of you will take abstract algebra, and part of it — group theory — is essentially the mathematics of symmetries. In that course you can prove there are only seventeen types of wallpaper: seventeen ways of making a pattern that repeats. That’s about wallpaper. But the same math applies to crystal structures — and, believe it or not, the same mathematics underlies the fundamental forces of nature. The forces of nature are expressions of symmetries, and the language we use for them is group theory, because it’s the same math. That’s the power of abstraction.

1.1.1 Working above and below the barrier

Why bring this up? Because you need to be able to work comfortably both above and below an **abstraction barrier**. If you go work at a company, you’re not going to know every detail of everything that’s happening. There’s a certain amount you can know, and for the rest you trust someone else’s expertise. There’s an abstraction barrier between you — and you represent an abstraction barrier to them, too: they don’t understand the data science and machine learning you’re doing.

Being able to understand what you need to know, and to be comfortable with what you don’t know, is the skill. It’s intimidating at first, when you feel like you don’t know anything. What I want you to come away with is confidence in what you do know and in your ability to figure things out. This is not a job where every day you turn the same crank. You have to be able to walk into a room knowing nothing about the thing in front of you and say: *I will figure this out. I can do that because I’ve done it many times.*

As a data scientist you can get away with using a library without knowing what’s inside it — but that will limit how good you are. That’s why, throughout this book, we build little versions of software you’d normally just import: seeing how it works underneath teaches you to use it better. It’s like driving a car. If you know nothing about what’s under the hood you can still drive, but if you understand roughly what the engine is doing, you’re a better driver.

This whole chapter works the same way (Figure 1). We start with electrons in wires. Each time we solve a problem, we draw an abstraction barrier around it, and then solve higher-level problems on top — over and over, from charge all the way to Python.

1.2 From Charge to Bits

I’m a physicist, so let me start where a physicist starts. One of the fundamental forces of nature is electricity and magnetism. It’s what keeps atoms together, and it’s responsible for all of chemistry — this, plus quantum mechanics, is essentially the foundation of atoms, molecules, and almost everything you touch. It’s also the foundation of electronics.

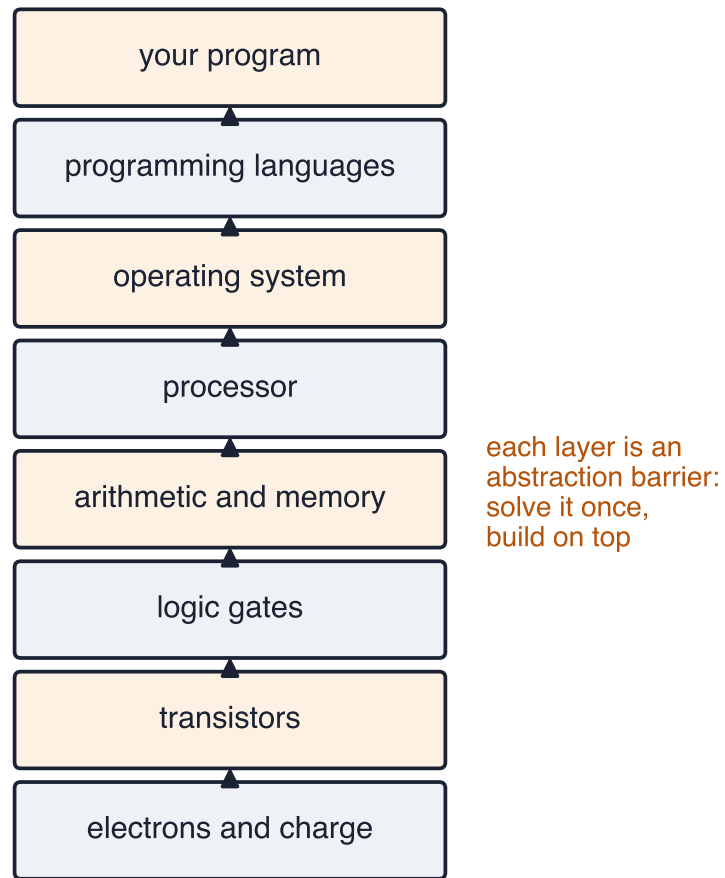


Figure 1.1: The abstraction ladder: each layer solves its problem once and becomes a building block for the next.

Electrons carry an **electric charge**. A charge produces an **electric field**, and it feels a force when it sits in another charge's field: like charges repel, opposite charges attract (Figure 12). That mutual push and pull is the whole engine.

Here's the idea of electronics. Charges can move — a **current**. In **insulators** they stay put; in **conductors** they move freely. How do you make them move? You set up an electric potential — a **voltage**. Think of a wire as a pipe and the electrons as water flowing through it: the voltage is the pressure pushing the water along. Close a switch and the loop completes — current flows and the bulb lights. Open it, and everything stops. There are other components in the toolbox too — **resistors** that impede the flow, capacitors that store charge, inductors — and you can combine elements in **series**, one after another along a single path, or in **parallel**, side by side on separate branches. Keep those two arrangements in mind; they're about to become logic.

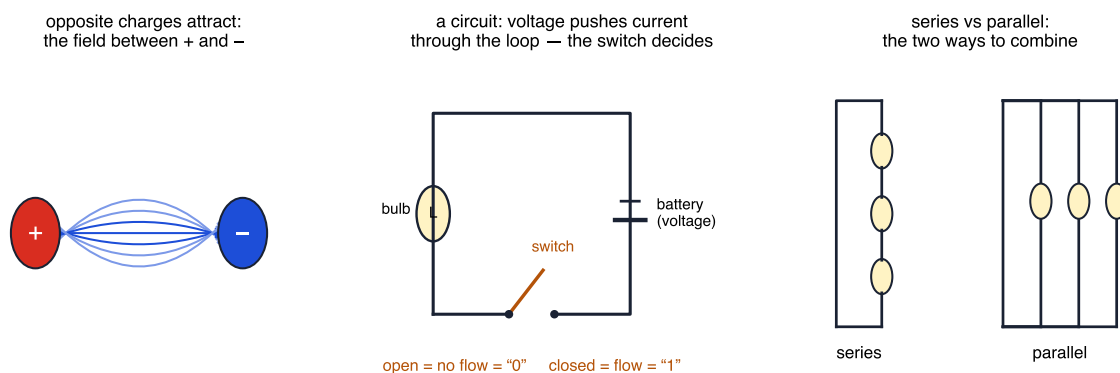


Figure 1.2: Charge and circuits: the field between opposite charges; a switch deciding whether current flows; series and parallel — the two ways to combine.

Now, we want to take computation — the calculations we, as abstract thinkers, do on paper — and represent it in a machine. Start with the simplest possible idea: zeros and ones. In an electronic computer, a zero is no flow of electrons in the pipe, and a one is a flow. And to know *when* to look, we add a **clock**: every tick, look at the wire and ask whether it's a zero or a one. That's the whole basic concept. Everything else is built from it.

1.2.1 The transistor

The next thing I need is a way to turn the flow on and off — an electronic switch. That's exactly what a **transistor** is (Figure 2).

A transistor has three terminals, and the names are worth knowing because every circuit diagram uses them. The **collector** is where current arrives. The **emitter** is where it leaves, on its way to the rest of your circuit. And the **base** is the control: current is allowed to flow from collector to emitter only while a small current is applied to the base. Base on, current flows; base off, nothing. It's a switch — that's all it is.

That's our first abstraction barrier. Below it: electrons, voltages, semiconductor physics. Above it: a thing that switches. From here on, we never think about the electrons again.

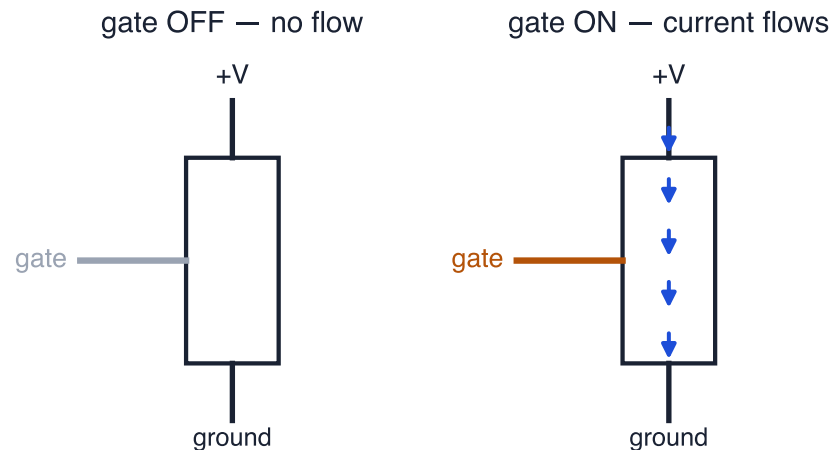


Figure 1.3: A transistor is an electronic switch: the base controls whether current may pass from collector to emitter.

1.3 Logic Gates

Now we've got a switch. Put switches together and we can start doing logic — and remember series and parallel from a moment ago, because that's exactly what comes back.

Suppose I want an output that turns on only when *both* inputs are on. Put two transistors in **series** — one after the other on the path from voltage to output. Current gets through only if both are switched on. That's an **AND**. Put the two transistors in **parallel** instead, and current gets through if *either* one is on — that's an **OR**. Take the output from the *top* of the circuit instead of the bottom and the logic inverts: series transistors give you **NAND** (on *unless* both inputs are on), parallel ones give you **NOR** (Figure 13).

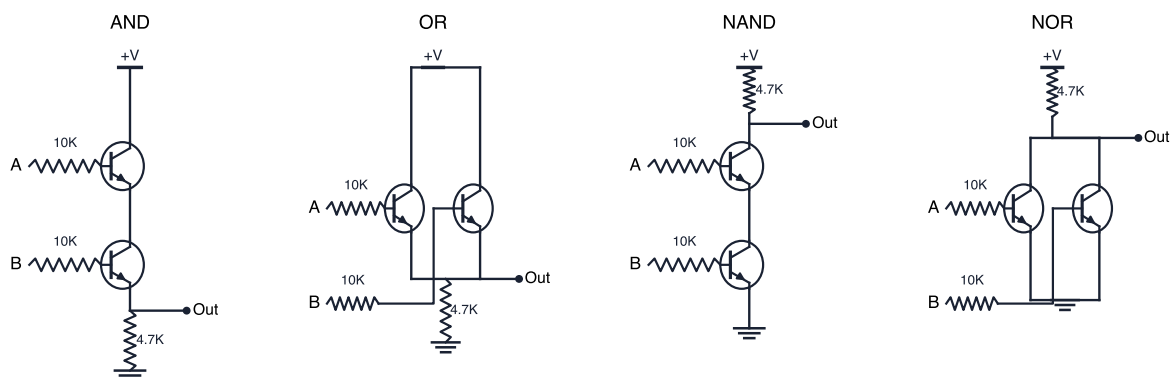


Figure 1.4: The four basic gates at the transistor level: series for AND-family, parallel for OR-family; where you tap the output decides whether the logic is inverted.

1.3.1 Walking the current through a gate

I want to walk through one of these slowly, because this is the step where “a pile of switches” turns into “logic,” and it deserves more than a hand-wave. Take the NOR circuit — two transistors in parallel, a resistor up to the positive voltage, output tapped at the top (Figure 14).

Start with both inputs off. Neither transistor conducts, so the charge coming down from +V through the resistor has nowhere to drain — no path to ground exists. The only place for it to go is out the output wire. The output sits at high voltage: $Q = 1$.

Now switch input A on. A’s transistor starts conducting, and suddenly there *is* a path to ground — right through it. Charge takes the easy way out; the output node is drained down to ground’s level. $Q = 0$. Switch B on too and nothing changes — two paths to ground drain just as well as one. $Q = 0$.

Read the results back: 1 only when both inputs are 0. That’s the NOR truth table — and notice that nobody *programmed* it. The table is just what the electrons do, given this arrangement of parts. Do the same walk on the series circuit and you’ll get AND. That’s the whole trick of digital electronics, seen once at full magnification: **the truth table is the circuit’s behavior, not a rule imposed on it.**

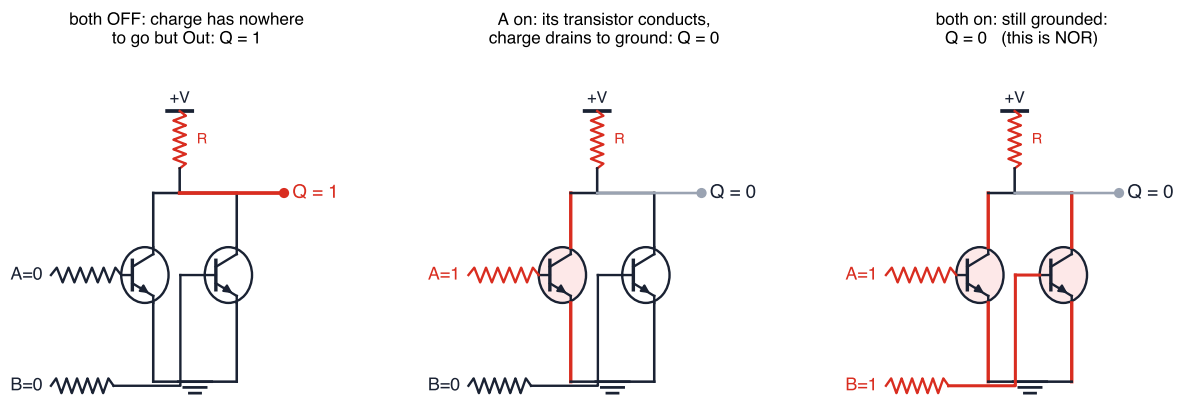


Figure 1.5: Walking the current through a NOR gate: with both inputs off the charge can only exit at Q; turning either input on opens a drain to ground and Q collapses to 0.

You won’t need to design circuits in this course, and if this section went by fast, the one thing to keep is the idea in bold above. But if you enjoyed it — this is the door the whole field of electrical engineering is behind.

1.3.2 The complete set

There are seven Boolean operations worth naming: **NOT**, **AND**, **OR**, **NAND**, **NOR**, **XOR** (“exclusive or” — on when the inputs *differ*), and **XNOR** (its opposite). Every one of them is a small transistor circuit like the ones you just saw (Figure 15).

NOT	AND	OR	NAND	NOR	XOR	XNOR																																																																																																																																	
<table><tr><td>A</td><td> </td><td>Q</td></tr><tr><td>0</td><td> </td><td>1</td></tr><tr><td>1</td><td> </td><td>0</td></tr></table>	A		Q	0		1	1		0	<table><tr><td>A</td><td>B</td><td> </td><td>Q</td></tr><tr><td>0</td><td>0</td><td> </td><td>0</td></tr><tr><td>0</td><td>1</td><td> </td><td>0</td></tr><tr><td>1</td><td>0</td><td> </td><td>0</td></tr><tr><td>1</td><td>1</td><td> </td><td>1</td></tr></table>	A	B		Q	0	0		0	0	1		0	1	0		0	1	1		1	<table><tr><td>A</td><td>B</td><td> </td><td>Q</td></tr><tr><td>0</td><td>0</td><td> </td><td>0</td></tr><tr><td>0</td><td>1</td><td> </td><td>1</td></tr><tr><td>1</td><td>0</td><td> </td><td>1</td></tr><tr><td>1</td><td>1</td><td> </td><td>1</td></tr></table>	A	B		Q	0	0		0	0	1		1	1	0		1	1	1		1	<table><tr><td>A</td><td>B</td><td> </td><td>Q</td></tr><tr><td>0</td><td>0</td><td> </td><td>1</td></tr><tr><td>0</td><td>1</td><td> </td><td>1</td></tr><tr><td>1</td><td>0</td><td> </td><td>1</td></tr><tr><td>1</td><td>1</td><td> </td><td>0</td></tr></table>	A	B		Q	0	0		1	0	1		1	1	0		1	1	1		0	<table><tr><td>A</td><td>B</td><td> </td><td>Q</td></tr><tr><td>0</td><td>0</td><td> </td><td>1</td></tr><tr><td>0</td><td>1</td><td> </td><td>0</td></tr><tr><td>1</td><td>0</td><td> </td><td>0</td></tr><tr><td>1</td><td>1</td><td> </td><td>0</td></tr></table>	A	B		Q	0	0		1	0	1		0	1	0		0	1	1		0	<table><tr><td>A</td><td>B</td><td> </td><td>Q</td></tr><tr><td>0</td><td>0</td><td> </td><td>0</td></tr><tr><td>0</td><td>1</td><td> </td><td>1</td></tr><tr><td>1</td><td>0</td><td> </td><td>1</td></tr><tr><td>1</td><td>1</td><td> </td><td>0</td></tr></table>	A	B		Q	0	0		0	0	1		1	1	0		1	1	1		0	<table><tr><td>A</td><td>B</td><td> </td><td>Q</td></tr><tr><td>0</td><td>0</td><td> </td><td>1</td></tr><tr><td>0</td><td>1</td><td> </td><td>0</td></tr><tr><td>1</td><td>0</td><td> </td><td>0</td></tr><tr><td>1</td><td>1</td><td> </td><td>1</td></tr></table>	A	B		Q	0	0		1	0	1		0	1	0		0	1	1		1
A		Q																																																																																																																																					
0		1																																																																																																																																					
1		0																																																																																																																																					
A	B		Q																																																																																																																																				
0	0		0																																																																																																																																				
0	1		0																																																																																																																																				
1	0		0																																																																																																																																				
1	1		1																																																																																																																																				
A	B		Q																																																																																																																																				
0	0		0																																																																																																																																				
0	1		1																																																																																																																																				
1	0		1																																																																																																																																				
1	1		1																																																																																																																																				
A	B		Q																																																																																																																																				
0	0		1																																																																																																																																				
0	1		1																																																																																																																																				
1	0		1																																																																																																																																				
1	1		0																																																																																																																																				
A	B		Q																																																																																																																																				
0	0		1																																																																																																																																				
0	1		0																																																																																																																																				
1	0		0																																																																																																																																				
1	1		0																																																																																																																																				
A	B		Q																																																																																																																																				
0	0		0																																																																																																																																				
0	1		1																																																																																																																																				
1	0		1																																																																																																																																				
1	1		0																																																																																																																																				
A	B		Q																																																																																																																																				
0	0		1																																																																																																																																				
0	1		0																																																																																																																																				
1	0		0																																																																																																																																				
1	1		1																																																																																																																																				

seven operations — every one is a small transistor circuit; from here on, just symbols

Figure 1.6: The seven Boolean operations. Every column is a small transistor circuit; from here on we use only the symbols.

And now I'm going to invoke abstraction again. I'm not drawing circuits anymore. Each operation gets a symbol, and whatever transistor arrangement sits behind the symbol, I no longer care. One layer down, sealed off.

1.4 Representation

The gates give us Boolean logic — true and false. But we want to do *math*, and for that I have to represent numbers as zeros and ones. You know this is called **binary**, though in my experience very few people have actually thought it through.

First, a word about **representation** itself, because it comes up everywhere. Language is a representation of ideas. Writing is a representation of language; so is speech; so is a recording. The same thing, represented several ways. Later, when you work with data, remember this: the same data can be represented in multiple ways, and the choice matters.

In fact, this is how large language models work: they map concepts into high-dimensional vector spaces where similar things sit near each other. They represent knowledge geometrically. We'll come back to that much later in the book.

1.4.1 Binary numbers

Here's how we represent integers using eight **bits** — eight zero-or-one values (Figure 4). Each position carries a value: the rightmost is $2^0 = 1$, the next $2^1 = 2$, then 4, 8, 16, and so on. To decode a number, multiply each bit by its position's value and add. The pattern 0 1 0 1 1 0 1 1 is $64 + 16 + 8 + 2 + 1 = 91$.

That's exactly how ordinary base-ten numbers work — it's the same system, just with two symbols instead of ten. Counting rolls over the same way, too: each position can only reach one, so incrementing past it carries into the next position, just as nine rolls to zero with a carry in decimal.

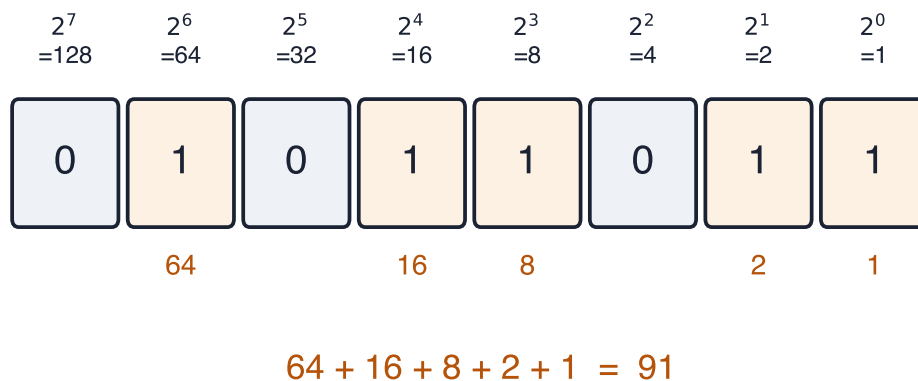


Figure 1.7: Eight bits, read as powers of two. Highlighted positions are “on” and contribute their value.

Eight bits give $2^8 = 256$ distinct patterns, so you can count from 0 to 255 — or, if you spend one pattern’s worth on the sign, from -128 to $+127$. Those are the two standard uses of a **byte**.

1.5 Arithmetic from Logic

Now the payoff. Here’s how a computer adds (Figure 5).

Start with a **half adder**, which adds two bits. Zero plus zero is zero; zero plus one is one; one plus one is *two* — which in binary is 10: a zero, plus a carry.

Look at what that requires. The sum bit is on when the inputs *differ* — that’s exactly XOR. The carry bit is on when *both* inputs are on — that’s exactly AND. One XOR gate, one AND gate, and you’ve added two bits.

A half adder handles the lowest column, but every other column of an addition has *three* things coming in: the two bits, plus the carry from the column before. The circuit for that is a **full adder** — built from two half adders and an OR gate — and its table covers all eight cases, from $0+0+0$ up to $1+1+1 = 11$ (Figure 16). To add numbers of any width, chain full adders: each column’s carry feeds the next, rippling left just like carrying in pencil-and-paper arithmetic. Work through it yourself once — it’s the kind of thing that’s hard to see all at once and obvious afterward. Similar circuits do the other operations.

So, fundamentally: electrons flow $\rightarrow$ transistors switch $\rightarrow$ gates do logic $\rightarrow$ gates arranged carefully do *arithmetic on our representation*. Nothing mystical has entered the machine at any step.

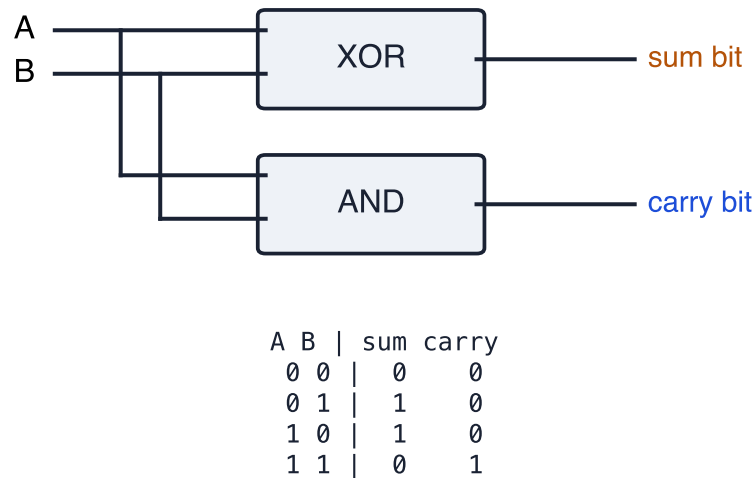


Figure 1.8: A half adder: XOR produces the sum bit, AND produces the carry. Two gates add two bits.

half adder: 2 bits

$0+0 = 00$
 $0+1 = 01$
 $1+0 = 01$
 $1+1 = 10$

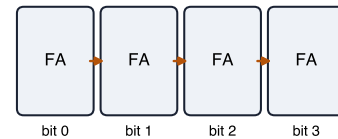
sum = XOR carry = AND

full adder: 2 bits + carry in

$0+0+0 = 00$ $1+1+0 = 10$
 $0+1+0 = 01$ $1+0+1 = 10$
 $1+0+0 = 01$ $0+1+1 = 10$
 $0+0+1 = 01$ $1+1+1 = 11$

two half adders + an OR

N-bit adder: chain the carries



carry ripples left; any width you like

Figure 1.9: From half adder to full adder to an N-bit adder: each column's carry ripples into the next.

1.6 Memory

Two more ingredients and we can build a computer.

First, storing information. There's a circuit called a **flip-flop** that remembers a value: tell it “you're a one,” and next time you ask, it says one. And here's the satisfying part — you already own the parts. One way to build a flip-flop is two NOR gates *feeding each other*: each gate's output wired into the other's input (Figure 17). That loop is the memory. Whichever state the pair falls into, each gate holds the other there, and the bit persists until you deliberately flip it. Storage isn't a new ingredient; it's the same gates, arranged to bite their own tail. (Modern RAM at scale uses an even smaller cell — one transistor holding charge on a capacitor — but the principle stands: remembering is a circuit trick.)

Second, a **multiplexer** — essentially a router. It has two inputs, a select line, and one output: if select is 0, input 0 comes through; if 1, input 1. Feed the select line a different kind of meaning and the same part picks between *operations* — plus or minus — which is exactly what it will do inside the processor in a moment.

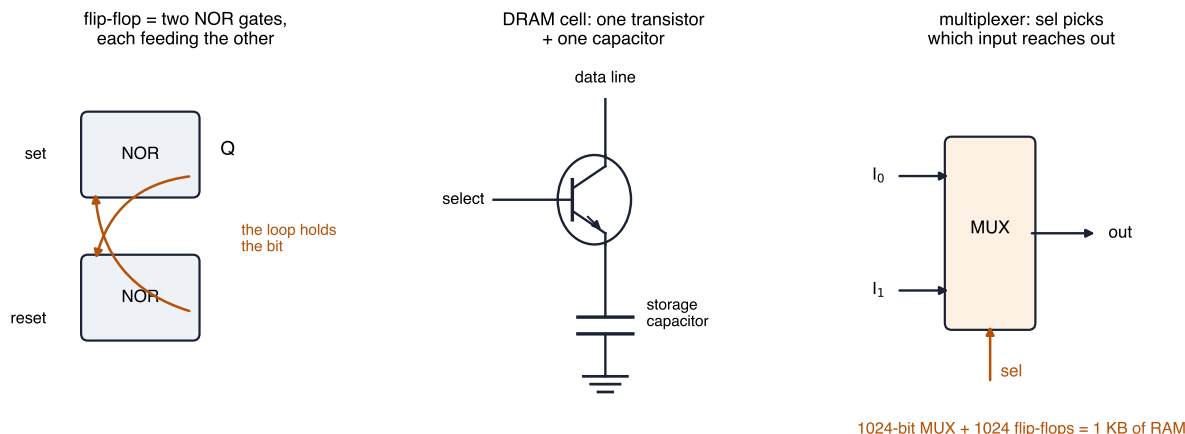


Figure 1.10: Inside memory: a flip-flop is two NOR gates holding each other in place; a DRAM cell is a transistor guarding a capacitor; a multiplexer routes by its select line.

Put them together: a bit arrives, the multiplexer decides *which* flip-flop it goes to, and that value is stored. That's **memory**. A large multiplexer with a large bank of flip-flops behind it is **random access memory (RAM)** — “random access” because you can jump straight to any address rather than reading in order. To put numbers on it: a 1,024-way multiplexer in front of 1,024 flip-flops is one kilobyte of RAM. Your laptop's sixteen gigabytes are that idea, scaled by ten million.

1.7 From a Calculator to a Computer

Let's build something with all this: a push-button calculator.

I need somewhere to hold numbers while I work on them. I'll call that a **register** — just a row of flip-flops, holding one eight-bit number. Give me three of them.

You press **8**: the binary pattern for 8 loads into the first register. You press **+**: a multiplexer connects the registers to the addition circuit. You press **3**: the pattern for 3 loads into the second register. You press **=**: the values push through the adder and the result lands in the third register. That’s a calculator, and you genuinely could build one from the components on these pages.

But a calculator needs you to push the buttons — *you* are the thing deciding what happens next. To get a computer, I need two more ideas. First, represent the operations themselves — plus, minus, compare — as numbers too: **instructions**, sitting in memory right alongside the data. Second, the clock: every tick, do the next thing, automatically, like pressing “=” forever.

That’s the **von Neumann architecture** (Figure 6) — the design of essentially every computer since the 1940s. Memory holds both instructions and data. A **control unit** keeps a **program counter** — which instruction am I on? — and each cycle it fetches that instruction, hands the work to the **arithmetic logic unit**, and moves to the next. Each instruction says: take the numbers at these two addresses, do this operation, put the result at that address.

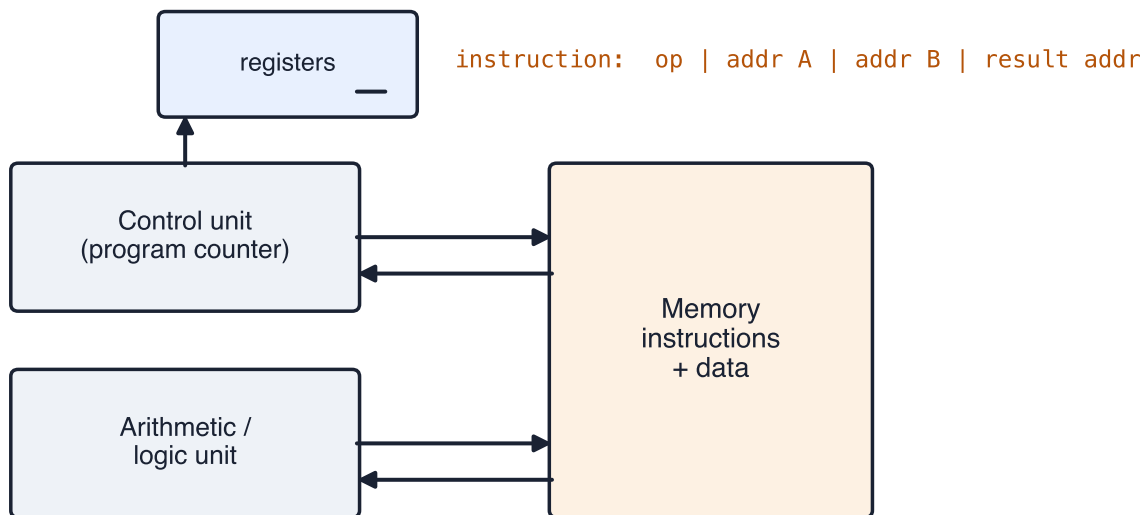


Figure 1.11: The von Neumann architecture: one memory holds both instructions and data; the control unit fetches and executes them one cycle at a time.

One special kind of instruction changes the program counter itself — jump back five instructions, or jump *only if* this number is bigger than that one. That’s control flow, and it’s what makes a program more than a list.

With those pieces, you have a computer. Modern machines are enormously more sophisticated — but I hope some of the magic and mystery is gone now. It all comes from electrons flowing. A modern processor is billions and billions of transistors doing exactly what’s on these pages.

1.8 How Chips Are Made

So how do you physically build billions of transistors? The technique is **lithography**: draw the circuit you want, and project it — like a projector — onto a silicon surface coated with photosensitive material. Where the light lands, chemistry happens: deposit metal, etch material away. Repeat, layer by layer. You’re literally printing transistors onto silicon with light.

For a long stretch, that progress was something you could read off an advertisement. A computer would be sold as an “Intel 386, 25 megahertz.” The 386 was the processor generation — a 32-bit machine, where the one before was 16-bit — and 25 megahertz was the clock: twenty-five million ticks per second. The next year the same shop would sell you a 33 megahertz one, and the year after that something faster still. Every time a switch flips, electrons flow and heat comes out; each new manufacturing process could be driven a little harder before it cooked itself.

That worked until the late nineties, at around four gigahertz — and then it stopped. Pushing clocks past that meant liquid-nitrogen territory: not practical. What kept improving instead was lithography. Structures got smaller, and *smaller structures need less power*. That’s why chips are now at the three-nanometer frontier, why each generation is more efficient — and why the strategy changed from “raise the clock” to “add more transistors.” Instead of one computation core per chip, we now put in many: **multicore** processors. One computer is effectively several.

The real constraints today are energy and heat. The same M-series chip sits in an iPad and in a Mac Pro — the difference is how much power the enclosure can feed it and how much heat it can carry away.

1.8.1 Caches and the memory bottleneck

Look at a processor die and you’ll see more than cores: there’s **cache** and a memory controller. Memory has historically lived on separate chips, and going off-chip to fetch data costs time and energy. So processors keep caches — small, fast, local copies. Fetch a block at once, and if you’re about to read a hundred values in a row, they’re already there (Figure 7). There are several levels, trading size against speed.

Two more tricks of modern processors are worth naming. Clock rates are **variable** now — the chip speeds up under load and slows down to save power and heat, which is why a laptop’s fan and its benchmark numbers both depend on what else is running. And processors do **out-of-order processing**: rather than idle while one instruction waits on memory, the chip quietly reorders nearby work to keep itself busy, preserving the *appearance* of executing your program in order.

One of the interesting things about Apple’s M-series chips is that the memory is stacked directly on the processor package. You can’t upgrade it afterward — but it’s *close*, and close is fast. And these chips carry specialized processors alongside the general cores: graphics processing units, neural engines. A gaming PC constantly copies data between CPU memory and GPU memory; on a chip where everyone shares one memory, that copying vanishes. That’s a large part of why Apple silicon performs the way it does.

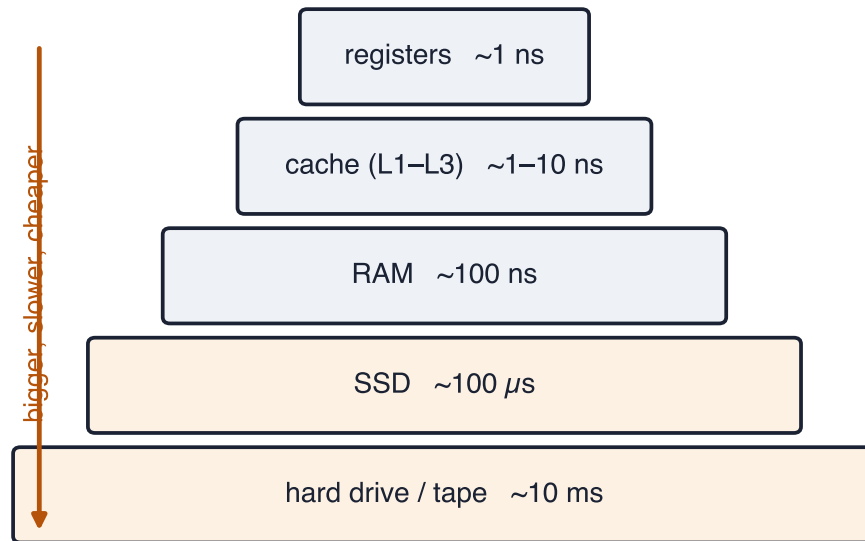
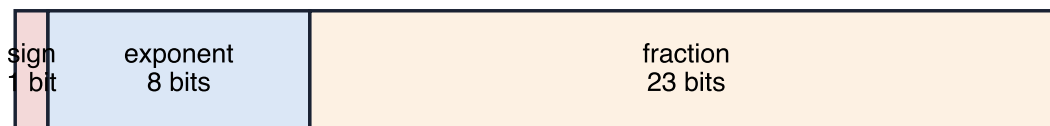


Figure 1.12: The memory hierarchy: each level trades capacity for speed. The processor works from the top; data migrates up as needed.

1.9 Floating Point

So far, integers. But science needs 3.14159 and 6.02×10^{23} . The representation for that is **floating point** — essentially scientific notation in binary (Figure 8). A 32-bit float spends 1 bit on the sign, 8 on the exponent, and 23 on the fraction. Most of the time you’ll use 32-bit or 64-bit floats without thinking about it.



$$\text{value} = (-1)^{\text{sign}} \times 1.\text{fraction} \times 2^{\text{exponent} - 127}$$

Figure 1.13: The anatomy of a 32-bit float: sign, exponent, fraction — scientific notation in binary.

(A preview of things to come: neural networks trained in 32-bit can often be *quantized* down to far fewer bits and still work. Keep that in your pocket for later chapters.)

Processors get rated by how fast they do this arithmetic: **FLOPS**, floating point operations per second — cores, times clock ticks per second, times operations per tick. The GPUs driving modern AI run *teraflops*: trillions per second. When you hear that a company is buying nuclear

power plants to feed data centers, this is the number they’re feeding.

1.10 Storage

Everything above — registers, cache, RAM — works like flip-flops: values held while the power is on. Turn the machine off and it all evaporates. For anything permanent you need **storage**.

For decades that meant magnetism. Tape stored bits as the orientation of a magnetic field; floppy disks spun the same idea on a disk; **hard drives** stacked rigid platters and packed the bits ever denser. Within living memory a *hundred-megabyte* hard drive was a serious purchase; today’s drives hold terabytes.

To use a disk you have to organize it. The disk is divided into **sectors** — the minimum unit you read or write, with an address corresponding to a physical location. Formatting a disk is essentially drawing that grid on it.

The past decade brought **solid state drives (SSDs)**: storage on silicon, no moving parts, and the contents survive power-off. The difference is orders of magnitude — an SSD can read around ten gigabytes per second where a spinning disk struggles to reach one. Interestingly, that speed changes the *architecture* of programs: things you’d have kept in RAM because disk was slow can now live on the SSD. At laptop scale we use SSDs everywhere; at data-center scale, spinning disks and tape are still far cheaper per byte, so the big, cold data lives there.

1.10.1 Swap

Here’s something you don’t see but should know about. Memory is expensive, and when the operating system runs out, it takes blocks of memory and writes them to disk to free up space — then reads them back when they’re needed. That’s **swap**. Usually it’s invisible. But if you’re running low on memory, the system starts swapping constantly, and everything crawls — because every “memory” access is now a disk access. One day you’ll be sitting there asking why your analysis got slow, and this will be the answer.

1.11 File Systems

A disk full of addressed sectors still isn’t *files*. One partition up from the hardware, we need organization.

First, a **partition** splits one physical disk into regions, each of which can carry its own system of organization — that’s how one machine dual-boots two operating systems. In Unix you can actually see these: `ls /dev` shows devices like `hda` and its first partition `hda1`. (Students ask: is WSL a partition? No — it lives inside a single file that emulates a whole file system. Hold that thought until the end of the chapter.)

Within a partition lives a **file system**. Here’s a thing we take for granted that somebody had to invent: the **file**. Bits on a disk are just bits; a file is the *idea* that these bits together are one thing, with a name. The file system keeps a **file table** mapping names to blocks: *this*

file lives at *these* addresses, created *then*, readable by *these people*. The data sits elsewhere, at those addresses. When a file doesn't fit contiguously, the file system splits it invisibly across free blocks — you never notice.

Files are grouped into directories, and directories nest: the hierarchical file system. Familiar, obvious — and also invented. There's a famous story about Steve Jobs touring Xerox PARC and seeing the pieces of modern computing — the graphical interface, the mouse, Ethernet — assembled in a research lab that never shipped them. Concepts have inventors, even the ones that feel like air.

Extensions — the `.pdf`, `.csv`, `.ipynb` on the end of names — are mostly convention rather than enforcement, but useful convention: they tell you and your tools what kind of thing a file claims to be. And **permissions**, enforced by the file system, say who may read or write what. You'll meet them the first time a command refuses to run.

1.12 Booting Up

You press the power button. What actually happens? (Figure 10.)

The processor wakes up knowing nothing, so the first instructions come from **firmware** — code written into a chip on the board, what PCs used to call the BIOS. These days it's flash and can be updated; it used to mean physically swapping chips. The firmware checks the hardware, brings up the peripherals, and then faces a problem: it has no idea what operating system you want. It reads a configured choice of drive and partition, loads the very beginning of it, and jumps in.

That very beginning is the **bootloader**, and it matters because of what *doesn't* exist yet: there's no file system, no operating system, nothing. The bootloader carries just enough information to locate the **kernel** — the core program of the operating system — read it into memory, and hand over control. If you've installed two operating systems, the bootloader is the menu that asks which one you want.

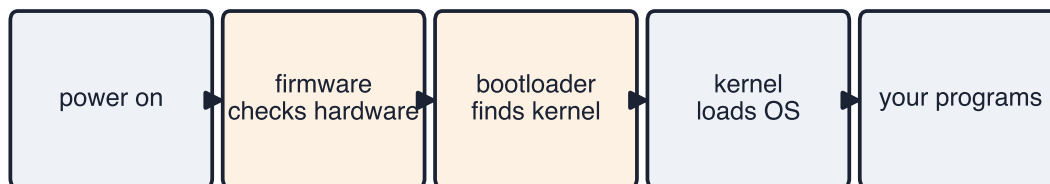


Figure 1.14: From power button to running programs: each stage knows just enough to find and start the next.

Firmware → bootloader → kernel → your programs. Each stage knows just enough to find the next.

1.13 Operating Systems

The **operating system** manages the computer’s hardware and software and provides common services for programs. There’s one processor, a fixed amount of memory, and many programs that all want to run. Something has to decide who gets what, and that’s the operating system. It’s the intermediary between programs and hardware.

If I want to save something to disk, I don’t tell the drive to write bits — I ask the operating system, and it deals with whatever hardware happens to be there. This is delivered through an **API** (application programming interface): a library of thousands of functions. When a Mac program pops up a window, it doesn’t draw the window; it *asks macOS for a window*. That’s also why software is built *for* an operating system — a program is largely a sequence of calls into one particular system’s API.

1.13.1 Multitasking

Modern systems all **multitask**, and not because you have many cores — even one core appears to run dozens of programs at once. The trick: run one program for a few cycles, take control back, run another, take control back. Each running program is a **process**, with its own allotment of memory. If nothing else wants the processor, your program gets nearly all of it; if many do, time is shared out.

The “take control back” part matters. **Preemptive multitasking** means the operating system *can* interrupt a program whether it cooperates or not — which is why one hung program no longer freezes the machine. Home computers only got this in the nineties; it arrived at the Mac by way of a story we’ll get to in a moment.

1.14 Unix

Unix was developed at Bell Labs in the early 1970s, alongside the C language — each built with the other. Computers then were mainframes: room-filling, wildly expensive, and shared. Many people connected at once through text terminals, so Unix had to be **multi-user** and multitasking *from birth*, with users and permissions baked in. The Apple II and DOS never needed any of that — one person, one floppy, run the program — and that difference in ancestry still shapes everything.

Licensing Unix was expensive, and in the 1980s two open-source implementations appeared: **Linux** and **BSD**. Both live on. And here’s the thing to understand: Unix today is really a *standard* — a philosophy of what an operating system should provide — and Linux, BSD, and friends are implementations of it. There is literally a certification for being “Unix-compliant.”

1.14.1 The family tree

The **Linux kernel** is just that — a kernel. The rest of a working system comes from other projects, most famously GNU. A **distribution** — Red Hat, Debian, Ubuntu — is someone

bundling kernel plus components into one installable package. That’s the whole mystery of “which Linux?”

macOS is *certified Unix*. When Steve Jobs was pushed out of Apple, his next company — NeXT — built its system on BSD. When Apple brought him back, the technology came too, and became Mac OS X. The kernel under your MacBook is a BSD descendant with Apple’s layers on top; iOS is the same system tuned for a phone.

Windows is a different lineage entirely: CP/M, adapted by Microsoft into DOS, evolving into Windows around 1985. It grew up on single-user home machines that needed none of the mainframe machinery, and though it has long since acquired those features, they were retrofitted. That’s why it feels — and *is* — structurally different from everything Unix-derived.

Why do I insist you learn Unix? Because besides Windows, it’s underneath nearly everything — every server, every supercomputer, every phone, every AI training run. If you’re going to work on any real system, it will be Unix-shaped, and being comfortable there is not optional. You’ll hit a wall in data science if you can’t touch a command line.

1.14.2 The Unix philosophy

A few principles explain most of what makes Unix pleasant to work in.

Text is the universal format. Configuration lives in plain text files a human can read and edit — not in binary formats only one program understands.

Everything is a file. Programs talk to devices, and to each other, through the same mechanism they use to read and write files. This sounds like trivia; it’s actually the superpower. Because a program can’t tell whether it’s reading a file, another program’s output, or a network stream — you can *chain* programs.

Small tools, composed. Listing files isn’t a feature built into the system — it’s a tiny program called `ls`. Deleting is `rm`; making a directory is `mkdir`. The **shell** — the command-line interpreter where you type commands — finds and runs them. Type a command, press enter, see the result. You’ve watched this in movies; soon you’ll do it yourself, and complicated work becomes small programs piped together on one line.

1.15 Networks

You all know about the internet. Let me give you a sense of how it actually works, because it’s layers of abstraction again, and by now you know how to read that (Figure 9).

At the bottom, the **physical layer**: computers connected by a wire (or radio — put the technology behind the barrier), able to put bits on it and see everyone else’s bits. One step up, bits are grouped into **frames** — everyone on the network agrees to talk in, say, hundred-bit chunks and take turns. Frames form **packets**, and a packet carries addresses: every network device on Earth has a unique ID, the **MAC address**, and a packet says who it’s from and who it’s for. Everyone watches the traffic; the machine whose address matches picks it up. There’s error checking, and a protocol for resending what got mangled — because plenty does.

That handles *one* network. To reach across the world, the **internet protocol (IP)** adds its own addressing: your machine picks up an IP address, and routers pass packets upward and onward, each one moving the packet closer, until it arrives. Since numbers are hard to remember, **DNS** — the domain name service — maps names to addresses: ask for a site by name, get back the address, connect. It’s a worldwide, synchronized phone book, fast enough that you never notice it.

One more layer. A single machine at one address might be doing a hundred jobs — so **TCP** adds **ports**: numbered doors on the machine. The web server traditionally listens behind door 80. Your browser asks for address *and* door, and the operating system routes the request to the right program.

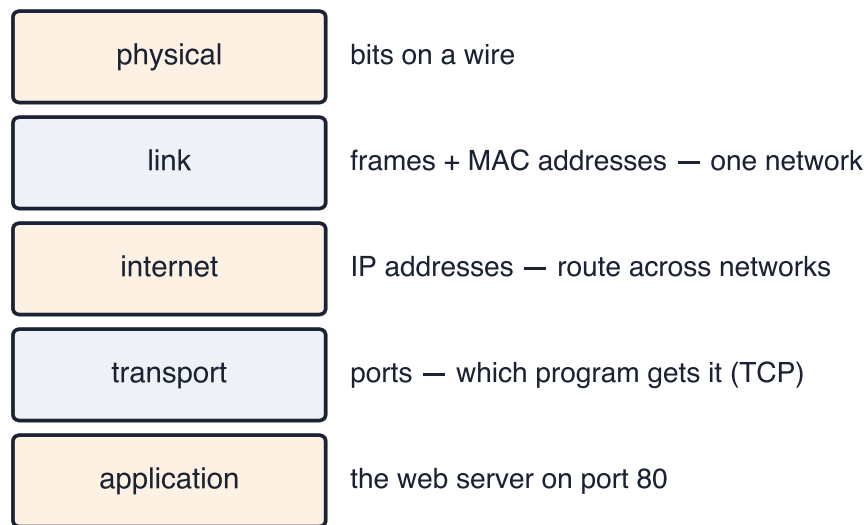


Figure 1.15: The network stack: each layer solves one problem and hands the rest upward.

And because you now understand DNS, you can understand an attack. Malware quietly infects thousands of machines; one day, on command, they all start hammering one company’s servers with requests. The server drowns, and the network behind it goes dark. That’s a **denial of service** attack — no code was “hacked”; the system was simply asked to do its job too many times at once.

1.16 Supercomputers and the Scale of Computing

A **supercomputer** is a huge number of computers on a very fast network, working in concert — thousands of processors and GPUs cooperating on one enormous calculation. Inside, the game is the same one from the chip section: put power in, take heat out.

The scale is changing under our feet. The machines governments fund for science cost hundreds of millions of dollars; companies are now building *billion*-dollar machines to train AI, and buying

nuclear power plants to feed the data centers. That is happening now, this year — and the reason, as you now know, is FLOPS per watt.

1.17 Running One System Inside Another

Now for the most practically useful section of this chapter — the ideas behind the tools you’ll use every week. This material rarely gets written down; it mostly lives in the heads of people who’ve been doing this a long time.

Every operating system has a different API. So I can’t take a program built for Linux and run it on Windows — and there’s a *second*, sneakier problem: a compiled program is machine language for a particular instruction set. An ARM chip (your phone, newer Macs) simply does not understand x86 instructions. Two incompatibilities, not one: the API and the architecture.

There are four approaches, and they solve different combinations (Figure 11).

Wrappers. Keep the program; translate its *calls*. A thin layer catches each function call meant for system A and invokes system B’s equivalent. This is how WSL 1 let Windows run Linux programs, how Wine runs Windows programs on Linux, and how SteamOS — which is Linux — runs an enormous Windows game catalog. For twenty years, needing Unix to get work done meant leaving Windows behind; that Microsoft now ships a Linux environment *inside* Windows tells you who won that argument.

Translation. Same operating system, different chip. When Apple moved from Intel to their own ARM chips, old programs were machine code for the wrong processor — so **Rosetta 2** translates the instructions themselves, x86 to ARM. Windows does the equivalent for x86 apps on ARM machines. It costs performance, but it works.

Virtual machines. Everything different? Then simulate an *entire computer* in software — CPU, storage, all of it — and install another operating system inside the simulation. The inner system believes it has a machine to itself. The simulated disk is just a file on the host, called an **image**. This is WSL 2: a real Linux, in a virtual machine, integrated into Windows.

And this is the cloud. Amazon’s hardware runs no normal operating system at all — it runs a **hypervisor**, a stripped-down system whose only job is running other people’s virtual machines. Ask for a server, and the hypervisor boots your image on a slice of hardware. “The cloud” is hypervisors all the way down.

Containers. A **container** is the lightweight version: don’t simulate a computer, just package an application together with everything it needs, on top of the existing kernel. Small, fast to start, disposable. When some software is miserable to install, the modern answer is a container that’s already configured. Every time you open a Google Colab notebook, a container just started somewhere. You’ve been using them all along.

For your own setup, know this: with WSL you are working with *two computers* — Windows on your laptop, and a Linux machine that happens to be virtual, one emulated inside the other. They can reach each other’s disks, but they are genuinely separate systems, and remembering that will save you confusion.

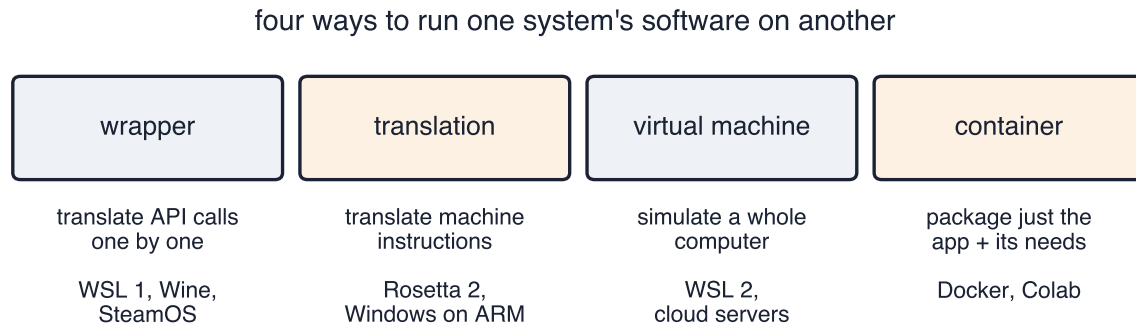


Figure 1.16: Four approaches to mixing systems, from translating single calls to simulating whole machines.

1.18 Programming Languages

Last layer of the ladder. Instructions in memory are bit patterns — “command number five.” **Assembly language** is those same commands with human-readable names, one thin abstraction up. You *can* write programs that way. In practice, nobody can build anything complex at that level.

So we write **high-level languages**, and they reach the machine two ways. A **compiled** language — C, C++, Java — is translated to machine language once, ahead of time; compiling may be slow, but the result runs at full speed. An **interpreted** language — Python — is read and executed on the fly by another program, the interpreter. That’s part of why Python is slower, and later in this book you’ll see the trick that saves us: Python orchestrating compiled libraries that do the heavy lifting.

1.18.1 What makes a language

MIT’s introductory computer science course used to open with a promise: learn three things, and you can learn *any* language. A programming language is three things.

Primitives — the atoms: numbers, characters, arithmetic, comparisons.

Means of combination — ways of putting things together: expressions built from expressions, lists of things.

Means of abstraction — there’s our word again — ways of naming things so they can be built upon: assignment, definitions, functions.

If a system has these three, it’s a programming language. Everything else is convenience. As you learn Python in the coming chapters, watch the concepts sort themselves into these three boxes

— and notice that the ideas you’re learning are not Python facts. They’re software development concepts. The language is just where we practice them.

Next: we put all of this to work. You’ll set up a Unix environment, meet the shell properly, and start building — because the fastest way to understand a computer is to make it do something.

Chapter 2

Glossary

Terms appear in the order a reader can use them; each was **bolded** at first appearance in the chapter.

abstraction — Dealing with ideas rather than their concrete details: once a problem is solved, treating it as a sealed input→output box and building on top of it.

abstraction barrier — The boundary around a solved problem. Work above it without re-deriving what’s below; open it only when the box itself is the problem.

charge — The property of matter that electric force acts on. Like charges repel; opposites attract.

insulator / conductor — Materials in which charge stays put / moves freely.

electric field — The influence a charge creates around itself; other charges feel a force in it.

current — Charge in motion through a conductor.

voltage — The electric potential that pushes charge through a conductor; the “pressure” in the water-pipe analogy.

resistor — A component that impedes current; in gate circuits, what keeps the rails from shorting and defines the output node.

series / parallel — The two ways to combine components: one after another on a single path, or side by side on separate branches. Series transistors make AND-family gates; parallel make OR-family.

clock — The regular tick that defines *when* a wire’s state counts as a 0 or a 1, and that steps a processor from one instruction to the next.

transistor — An electronic switch and the base unit of all digital electronics. Its three terminals: the **collector** (current in), the **emitter** (current out), and the **base** — the control that decides whether current may pass.

NOT, AND, OR, NAND, NOR, XOR, XNOR — The seven Boolean operations; each is a small transistor arrangement whose output is a fixed Boolean function of its inputs. XOR

(“exclusive or”) is on exactly when its inputs differ; XNOR is its opposite; NOT inverts a single input. The truth table is the circuit’s behavior, not a rule imposed on it.

binary — Positional base-2 number representation; each bit position is worth a power of two.

bit — A single 0-or-1 value.

byte — Eight bits; 256 distinct patterns (0–255 unsigned, –128 to +127 signed).

representation — A chosen encoding of information (ideas as language, numbers as bits, colors as pixel values). The same information can be represented many ways, and the choice matters.

half adder — The two-gate circuit (XOR for sum, AND for carry) that adds two bits.

full adder — Two half adders plus an OR gate: adds two bits *and* a carry in. Chained so each column’s carry ripples into the next, full adders add numbers of any width.

flip-flop — A circuit that remembers one bit until told otherwise: two NOR gates feeding each other, each holding the other in place. The loop is the memory.

multiplexer — A routing circuit: connects its input to one of several outputs on command.

memory / RAM — Flip-flops behind multiplexers: values stored at addresses, any of which can be reached directly (“random access”). Contents vanish at power-off.

register — A small, immediately accessible row of flip-flops where the processor holds the values it is actively working on.

instruction — An operation encoded as a number and stored in memory alongside data: what to do, which addresses to do it to, where to put the result.

von Neumann architecture — The standard computer design: one memory holding both instructions and data, a control unit stepping through them, and an arithmetic logic unit doing the work.

program counter — The control-unit register holding the address of the current instruction; writing to it (conditionally) is control flow.

arithmetic logic unit (ALU) — The circuit block that performs arithmetic and logic operations.

lithography — Chip fabrication by projecting circuit patterns onto photosensitive silicon and applying chemistry, layer by layer.

multicore — Several complete computation cores on one processor chip — the industry’s answer to the clock-speed/heat wall.

cache — Small, fast memory near the cores holding local copies of recently used data, avoiding slow trips to main memory. Arranged in levels (L1–L3).

floating point — Scientific notation in binary: sign, exponent, fraction. Trades exactness for enormous range.

FLOPS — Floating point operations per second; cores $\times$ clock $\times$ operations per cycle. The currency of scientific and AI computing.

storage — Persistent media (magnetic disk, tape, SSD) whose contents survive power-off, unlike RAM.

hard drive — Spinning magnetic platters; cheap per byte, slow, mechanical.

solid state drive (SSD) — Persistent storage on silicon; no moving parts, orders of magnitude faster than spinning disks.

sector — The minimum addressable read/write unit of a disk; formatting lays down the grid of sectors.

swap — The operating system’s use of disk as overflow memory: blocks are written out when RAM runs short and read back on demand — transparently, until it’s the reason everything is slow.

partition — A region of one physical disk organized independently, each able to carry its own file system.

file — The invented abstraction that a set of disk blocks, taken together, is one named thing.

file system — The organization within a partition: the file table mapping names to blocks, plus metadata and permissions.

permissions — File-system-enforced rules about who may read, write, or execute what.

firmware — Instructions stored in a chip on the board (once “BIOS”) that run first at power-on, check the hardware, and hand off to the bootloader.

bootloader — The tiny program at the start of a disk that runs before any file system exists, knows just enough to find the kernel, loads it, and hands over control.

kernel — The core program of an operating system — the thing the bootloader loads and everything else runs on top of.

operating system — The manager of hardware and software: shares processor, memory, and devices among programs and exposes services through an API.

API (application programming interface) — The set of functions programs call to ask the operating system (or any library) for services.

process — A running program plus its allotment of memory and resources, as managed by the operating system.

multitasking / preemptive multitasking — Rapidly switching the processor among processes. “Preemptive” means the OS can take control back — a hung program can’t freeze the machine.

multi-user — Designed from the start for several people using one machine, with users and permissions — Unix’s mainframe inheritance.

Unix — The 1970s Bell Labs operating system whose design became a standard; Linux, BSD, and macOS are implementations of its philosophy.

Linux kernel / distribution — The open-source Unix kernel / a packaged, installable bundle of kernel plus tools (Ubuntu, Debian, Red Hat).

BSD — The Berkeley open-source Unix; ancestor of the macOS kernel by way of NeXT.

shell — The command-line interpreter: type a command, it finds and runs the corresponding program, and shows the result. The primary Unix interface.

physical layer / frame / packet — Bits on a wire; a fixed-size chunk of bits everyone agrees to speak in; frames assembled with addressing into a deliverable unit.

MAC address — The globally unique hardware ID every network device carries; addressing within one network.

IP (internet protocol) — The addressing scheme that routes packets *between* networks, hop by hop, worldwide.

DNS (domain name service) — The distributed, synchronized mapping from names to IP addresses.

TCP / port — The transport protocol; ports are numbered doors on one machine so packets reach the right program (web servers traditionally answer on port 80).

denial of service — An attack by legitimate-looking volume: thousands of machines making ordinary requests until the target drowns.

supercomputer — Many computers on a very fast network computing in concert; the engineering problem is power in, heat out.

assembly language — Machine instructions with human-readable names; the thinnest abstraction above raw bits.

high-level language — A language written for humans and translated to machine language — either ahead of time or on the fly.

compiled / interpreted — Translated once ahead of time into machine code (C, C++) versus read and executed on the fly by an interpreter program (Python). The classic speed/convenience trade.

wrapper — A thin layer translating one operating system's API calls into another's (WSL 1, Wine, SteamOS).

translation (instruction) — Converting machine instructions from one processor architecture to another (Rosetta 2, Windows-on-ARM).

virtual machine — A complete computer simulated in software, running its own operating system; its disk is an **image** file on the host.

hypervisor — A stripped-down system whose only job is running virtual machines; what cloud hardware actually runs.

container — A lightweight packaging of one application plus its needs, sharing the host's kernel; near-instant to start (Docker, what backs a Colab session).

WSL (Windows Subsystem for Linux) — Microsoft's Linux environment inside Windows: version 1 was a wrapper, version 2 is a real Linux in an integrated virtual machine.

primitives / means of combination / means of abstraction — The three things that make any programming language: atoms, ways to put them together, and ways to name the result so you can build on it.

